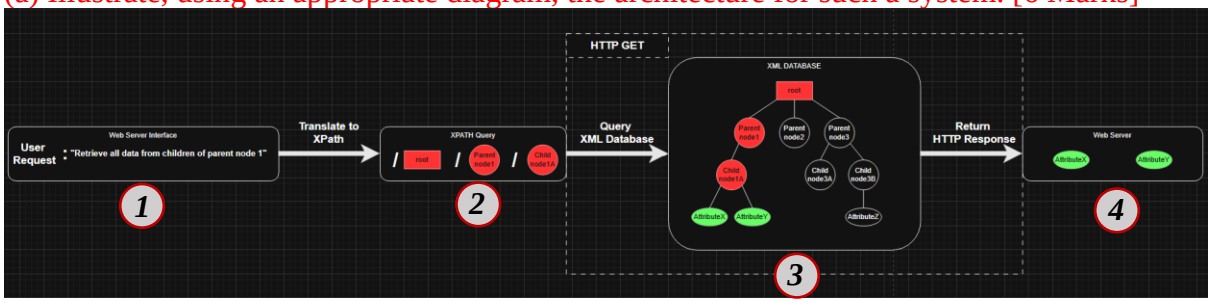
Q1 This question examines how XML and XPath may be used, when combined with a customised web server, to serve snippets of structured information over the Web.
(a) Illustrate, using an appropriate diagram, the architecture for such a system. [6 Marks]



1.	A human-readable query is submitted via the web interface.
2.	The user request is translated into a formal XPath expression (e.g., /root/ParentNode1/ChildNode1A) to target specific data in the XML database.
3.	The nodes selected by the query (red) are used to locate the attributes to be returned (green).
4.	The extracted attributes are returned by the server as a response.

(b) Develop an XML Schema Definition (XSD) to describe a new XML vocabulary for a movie or TV show production. This may include performance and logistical aspects, such as actors, characters, scenes, dialogue, props, scene instructions, producers, directors, etc., as you deem appropriate. The XSD should specify both elements and attributes. Element nesting and cardinality of some elements should also be included. Illustrate the use of as many features of XSD as you deem appropriate. [6 Marks]

```xml
<?xml version="1.0" encoding="UTF-8"?>

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" elementFormDefault="qualified">


    <!-- Root Element: AnimatedShow -->
    <xs:element name="AnimatedShow">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="Title" type="xs:string"/>
                <xs:element name="Season" type="xs:positiveInteger"/>
                <xs:element name="Episode" maxOccurs="unbounded">
                    <xs:complexType>
                        <xs:sequence>
                            <xs:element name="EpisodeTitle" type="xs:string"/>
                            <xs:element name="EpisodeNumber" type="xs:positiveInteger"/>
                            <xs:element name="Director" type="xs:string"/>
                            <xs:element name="Producer" maxOccurs="unbounded" type="xs:string"/>
                            <xs:element name="Scene" maxOccurs="unbounded">
                                <xs:complexType>
                                    <xs:sequence>
                                        <xs:element name="SceneNumber" type="xs:positiveInteger"/>
                                        <xs:element name="Description" type="xs:string"/>
                                        <xs:element name="Location" type="xs:string"/>
                                        <xs:element name="Shot" maxOccurs="unbounded">
```

```
                                        <xs:complexType>
                                            <xs:sequence>
                                                <xs:element name="ShotNumber" type="xs:positiveInteger"/>
                                                <xs:element name="ShotInstructions" type="xs:string" minOccurs="1"
maxOccurs="unbounded"/>
                                                <xs:element name="Prop" minOccurs="0" maxOccurs="unbounded">
                                                    <xs:complexType>
                                                        <xs:sequence>
                                                            <xs:element name="PropName" type="xs:string"/>
                                                            <xs:element name="Usage" type="xs:string"/>
                                                        </xs:sequence>
                                                    </xs:complexType>
                                                </xs:element>
                                                <xs:element name="Dialogue" maxOccurs="unbounded">
                                                    <xs:complexType>
                                                        <xs:sequence>
                                                            <xs:element name="Character" type="xs:string"/>
                                                            <xs:element name="Text" type="xs:string"/>
                                                        </xs:sequence>
                                                    </xs:complexType>
                                                </xs:element>
                                            </xs:sequence>
                                        </xs:complexType>
                                    </xs:element>
                                </xs:sequence>
                            </xs:complexType>
                        </xs:element>
                        <xs:element name="VoiceActor" maxOccurs="unbounded">
                            <xs:complexType>
                                <xs:sequence>
                                    <xs:element name="ActorName" type="xs:string"/>
                                    <xs:element name="CharacterName" type="xs:string"/>
                                </xs:sequence>
                            </xs:complexType>
                        </xs:element>
                    </xs:sequence>
                </xs:complexType>
            </xs:element>
        </xs:sequence>
    </xs:complexType>
</xs:element>


</xs:schema>
```

To Clarify:

Prop	Any non-character entities that move (/are supposed to be animated) during the scene.
Shot	Anytime there is a change in camera angle or position within a scene [panning excluded].
Scene	A series of shots that take place in a new location than the shot before it. [A building exterior shot to a building interior shot counts as two separate scenes. (i.e SceneA-Outside house & SceneB-Inside house)]

(c) Create an example XML document that is valid against your XSD (from part (b)) that illustrates as many features from the XSD as possible. [4 Marks]

```xml
<?xml version="1.0" encoding="UTF-8"?>

<AnimatedShow>

    <Title>Invincible</Title>

    <Season>1</Season>

    <Episode>

        <EpisodeTitle>It’s About Time</EpisodeTitle>

        <EpisodeNumber>1</EpisodeNumber>

        <Director>Jeff Allen</Director>

        <Producer>Simon Racioppa</Producer>

        <Producer>Robert Kirkman</Producer>

        <Scene>

            <SceneNumber>5</SceneNumber>

            <Description>The Grayson family sits at the dinner table. Mark is about to tell his parents the good news.</Description>

            <Location>Grayson Family Dining Room</Location>

            <Shot>

                <ShotNumber>1</ShotNumber>

                <ShotInstructions>Mark enters the dining room and sits at the table. Debbie finishes setting the table</ShotInstructions>

                <Prop>

                    <PropName>Plate of Roast Chicken</PropName>

                    <Usage>Held by Debbie. Placed on the table out of view for the rest of the scene.</Usage>

                </Prop>

                <Dialogue>

                    <Character>Debbie Grayson</Character>

                    <Text>And a big welcome home to my favorite son.</Text>

                </Dialogue>

            </Shot>

            <Shot>

                <ShotNumber>2</ShotNumber>

                <ShotInstructions>A close-up of Debbie as she sits at the table opposite to Mark</ShotInstructions>

                <Dialogue>

                    <Character>Debbie Grayson</Character>

                    <Text>I made some chicken, but I also heated up some authentic German bratwurst flown in special today, just for you.</Text>

                </Dialogue>

            </Shot>

            <Shot>

                <ShotNumber>3</ShotNumber>

                <ShotInstructions>A close-up of Mark sitting at the opposite side of the table</ShotInstructions>

                <Dialogue>

                    <Character>Mark Grayson</Character>

                    <Text>I... had a pretty interesting day.</Text>

                </Dialogue>

            </Shot>

            <Shot>

                <ShotNumber>4</ShotNumber>

                <ShotInstructions>Nolan arrives at the dinner table in a blur of motion, kisses Debbie on her forehead, and sits at the table.</ShotInstructions>

                <Prop>

                    <PropName>Wine Glass</PropName>
```

```
            <Usage>Almost tips over as Nolan enters the dining room. Is caught by Debbie and repositioned.</Usage>
        </Prop>
        <Dialogue>
            <Character>Nolan Grayson</Character>
            <Text>Sorry I'm late. Honest to God, a dragon was attacking Hong Kong. I stopped it.</Text>
        </Dialogue>
        <Dialogue>
            <Character>Debbie Grayson</Character>
            <Text>Ooh, a dragon. Nice. Mark was getting ready to tell us how interesting his day was.</Text>
        </Dialogue>
    </Shot>
    <Shot>
        <ShotNumber>5</ShotNumber>
        <ShotInstructions>Point of view from Noland and Debbie facing towards Mark.</ShotInstructions>
        <Dialogue>
            <Character>Mark Grayson</Character>
            <Text>Guess who's finally getting his powers? Haha.</Text>
        </Dialogue>
    </Shot>
    <Shot>
        <ShotNumber>6</ShotNumber>
        <ShotInstructions>Focus on Nolan's reaction as he processes Mark's news.</ShotInstructions>
        <Dialogue>
            <Character>Nolan Grayson</Character>
            <Text>Are you sure?</Text>
        </Dialogue>
    </Shot>
    <Shot>
        <ShotNumber>7</ShotNumber>
        <ShotInstructions>Focus on Mark as he responds, uncertain of his father's excitement.</ShotInstructions>
        <Dialogue>
            <Character>Mark Grayson</Character>
            <Text>Pretty sure. Threw a trash bag into space. At work.</Text>
        </Dialogue>
    </Shot>
    <Shot>
        <ShotNumber>8</ShotNumber>
        <ShotInstructions>Point of view behind Mark facing towards Nolan and Debbie.</ShotInstructions>
    </Shot>
    <Shot>
        <ShotNumber>9</ShotNumber>
        <ShotInstructions>Point of view from underneath the table. Debbie kicks Nolan's foot to regain his attention.</ShotInstructions>
    </Shot>
    <Shot>
        <ShotNumber>10</ShotNumber>
        <ShotInstructions>Noland, perks up to respond appropriately to the situation.</ShotInstructions>
        <Dialogue>
            <Character>Nolan Grayson</Character>
```

```
                    <Text>Oh, that's great, son, just great! If you're up for it, i'll make some time tomorrow for training.</Text>
                </Dialogue>
                <Dialogue>
                    <Character>Debbie Grayson</Character>
                    <Text>This is so exciting!</Text>
                </Dialogue>
                <Dialogue>
                    <Character>Nolan Grayson</Character>
                    <Text>Bright and early tomorrow, Mark. Make sure you get a good night's sleep.</Text>
                </Dialogue>
            </Shot>
            <Shot>
                <ShotNumber>11</ShotNumber>
                <ShotInstructions>Focus on Mark as he responds, elated at his father's response.</ShotInstructions>
                <Dialogue>
                    <Character>Mark Grayson</Character>
                    <Text>Ha! For sure!</Text>
                </Dialogue>
            </Shot>
        </Scene>
        <VoiceActor>
            <ActorName>Sandra Oh</ActorName>
            <CharacterName>Debbie Grayson</CharacterName>
        </VoiceActor>
        <VoiceActor>
            <ActorName>Steven Yeun</ActorName>
            <CharacterName>Mark Grayson</CharacterName>
        </VoiceActor>
        <VoiceActor>
            <ActorName>J.K. Simmons</ActorName>
            <CharacterName>Nolan Grayson</CharacterName>
        </VoiceActor>
    </Episode>
</AnimatedShow>
```

(d) Assume the customised web server can handle URLs of the form: http://your.server.net/?f=myfile.xml&q=xpath. The URL parameter myfile.xml represents a specific XML file (e.g. the one created in part (c)) and the parameter xpath is an XPath expression.
Give a step-by-step description of how a snippet of the XML file may be returned (via HTTP) based on the XPath expression. You may illustrate the steps using pseudocode, PHP, JSP, ASP, or similar, if you wish. [15 Marks]

1. Receive & Parse the HTTP Request

When a client sends an HTTP GET request to the following URL:

```
http://your.server.net/?f=Invincible.xml&q=/AnimatedShow/Episode/Scene/Shot[1]
```

, the web server begins by parsing the query string from the URL. It extracts the f parameter, which indicates the name of the XML file to be accessed (in this case, Invincible.xml), and the q parameter, which contains the XPath expression used to locate specified node(s) within that XML file.

```
function handleRequest(request):
    filename   = request.queryParams["f"]
    xpathQuery = request.queryParams["q"]
```

2. Input Validation

The server then validates the inputs. It ensures that the provided filename is safe to use, checking that it does not contain malicious patterns such as directory traversal sequences (../), and optionally verifies that the XPath expression is syntactically valid. If either check fails, the server can return an appropriate error response, such as HTTP 400 Bad Request.

```
    if not isValidFilename(filename):
        return HTTPResponse(400, "Invalid filename")
    if not isWellFormedXPath(xpathQuery):
        return HTTPResponse(400, "Invalid XPath")
```

3. Load and Parse the XML

Next, the server attempts to locate and open the XML file from a predefined directory. It reads the file's contents into memory and parses it into an in-memory XML DOM (Document Object Model) using a suitable parser. If the file is not found or contains malformed XML, the server returns an error, such as HTTP 404 Not Found or 500 Internal Server Error.

```
    xmlText = readFile("/data/xml/" + filename)
    try:
        xmlDoc = XMLParser.parse(xmlText)
    except ParseError:
        return HTTPResponse(500, "Malformed XML")
```

4. Evaluate the XPath Expression

With a valid XML document in memory, the server now evaluates the XPath expression against it. This involves compiling or interpreting the XPath and using it to search through the parsed XML tree. The result is a list of XML nodes that match the query. If the evaluation fails due to an invalid or unsupported XPath expression, the server responds with an error indicating the problem.

```
    xpathEngine = XPathEngine(xmlDoc)
    try:
        nodes = xpathEngine.evaluate(xpathQuery)
    except XPathError:
        return HTTPResponse(400, "XPath evaluation error")
```

5. Serialise the Resulting Snippet

If matching nodes are found, the server serialises them back into XML text. Depending on implementation, these nodes might be wrapped in a common root element for consistency or returned as-is. If no matches are found, the server may return an empty XML element to indicate that the query was valid but yielded no results.

```
    if nodes.isEmpty():
        body = "<result/>"
    else:
        body = ""
        for node in nodes:
            body += XMLSerializer.serialize(node)
```

6. Send the HTTP Response

Finally, the server constructs an HTTP response. It sets the status to 200 OK, includes appropriate headers (such as Content-Type: application/xml; charset=UTF-8), and places the serialised XML snippet into the response body. This response is then sent back to the client, who receives only the portion of the XML document specified by the original XPath query.

```
    return HTTPResponse(
        status=200,
        headers={"Content-Type":"application/xml; charset=UTF-8"},
        body=body
    )
```

(e) Illustrate a variety of features from the XPath specification, using at least four XPath expressions (xpath in part (d)). These expressions should extract information from your XML file (myfile.xml) and should illustrate how different features, e.g. node types, predicates, axes, are used. [4 Marks]

1. Selecting All Character Nodes in All Dialogue Elements

This expression demonstrates node selection. It extracts all the Character elements within the Dialogue elements, regardless of which Scene or Shot they are in.

```
//Dialogue/Character
```

//	Selects nodes anywhere in the document.
Dialogue	Selects all Dialogue elements.
Character	Selects all Character elements within Dialogue elements.

Returns all Character nodes from all Dialogue elements.

```
1. <Character>Debbie Grayson</Character>
2. <Character>Debbie Grayson</Character>
3. <Character>Mark Grayson</Character>
4. <Character>Nolan Grayson</Character>
5. <Character>Debbie Grayson</Character>
6. <Character>Mark Grayson</Character>
7. <Character>Nolan Grayson</Character>
8. <Character>Mark Grayson</Character>
9. <Character>Nolan Grayson</Character>
10. <Character>Debbie Grayson</Character>
11. <Character>Nolan Grayson</Character>
12. <Character>Mark Grayson</Character>
```

2. Selecting ShotInstructions of Shots Where ShotNumber is 1

This expression demonstrates predicates to filter nodes based on conditions. It selects the ShotInstructions for the first shot in a scene.

```
//Shot[ShotNumber=1]/ShotInstructions
```

//	Selects nodes anywhere in the document.
Shot	Selects all Shot elements.
[ShotNumber=1]	Predicate filters Shot elements where the ShotNumber is equal to 1.
ShotInstructions	Selects the ShotInstructions for that shot.

Returns the ShotInstructions for ShotNumber 1.

```
1. <ShotInstructions>Mark enters the dinning room and sits at the table.
   Debbie finishes setting the table</ShotInstructions>
```

3. Selecting the Text of Dialogue Spoken by a Specific Character

This example uses attributes and predicates to filter dialogue spoken by a specific character.

```
//Dialogue[Character="Mark Grayson"]/Text
```

//	Selects nodes anywhere in the document.
Dialogue	Selects all Dialogue elements.
[Character="Mark Grayson"]	Predicate filters the Dialogue elements where the Character is "Mark Grayson".
Text	Selects the Text of that Dialogue element.

Returns all text spoken by Mark Grayson.

```
1. <Text>I... had a pretty interesting day.</Text>
2. <Text>Guess who's finally getting his powers? Haha.</Text>
3. <Text>Pretty sure. Threw a trash bag into space. At work.</Text>
4. <Text>Ha! For sure!</Text>
```

4. Selecting the PropName of Props Used in the First Shot

This XPath expression demonstrates axis selection and position-based selection. It selects all PropName elements in the first Shot within a Scene.

```
//Scene/Shot[1]//Prop/PropName
```

//	Selects nodes anywhere in the document.
Scene	Selects all Scene elements.
Shot[1]	Selects the first Shot element within each Scene.
//Prop	Selects all Prop elements in the selected Shot.
PropName	Selects the PropName element within each Prop.

Returns the PropName of props used specifically in the first shot.

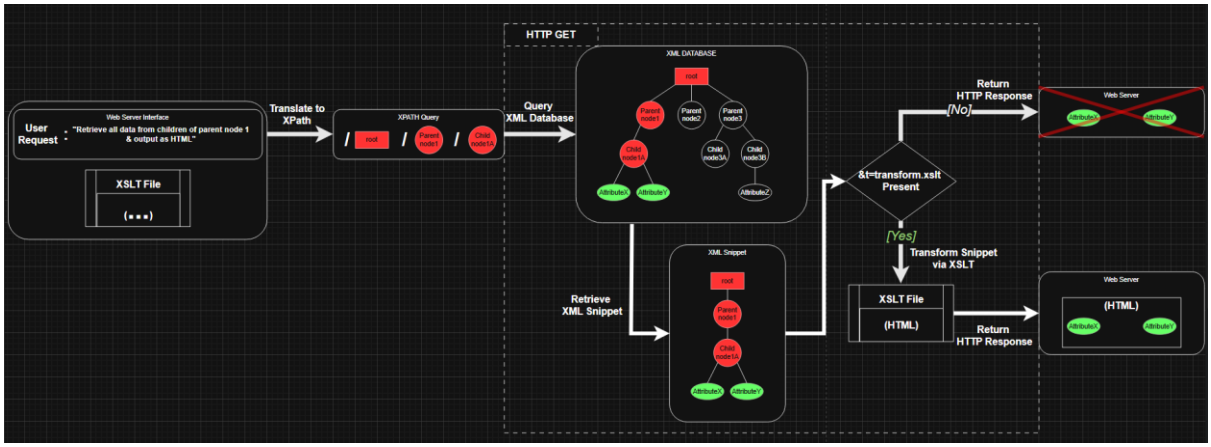
```
1. <PropName>Plate of Roast Chicken</PropName>
```

(Screenshots taken in https://xpather.com/)

(f) If another optional URL parameter, &t=transform.xslt, were added to the URL describe how the architecture (from part (a)) may change to handle transformation of the returned snippet. How would the step-by-step description from part (d) need to change? Give an example of an appropriate XSLT that might transform a snippet returned by one of your XPath expressions from part (e). Describe how that XSLT performs the transformation. [15 Marks]

Describe how the architecture (from part (a)) may change to handle transformation of the returned snippet.

The introduction of a &t=transform.xslt parameter would mean that the customised web server would have to be capable not only of parsing and querying XML via XPath, but also of applying an XSLT stylesheet to transform the resulting XML snippet before sending it back as an HTTP response. This is reflected in the updated architecture illustration for the customised web server as shown:



In such a case, the architecture of the web server is affected in the following ways:

1.New Logic Branch in the Web Server:

After the XPath query has been evaluated against the XML database and a snippet is retrieved, the server checks if the &t= parameter is present. If it is, the server will load the specified XSLT file and apply it to the XML result.

2.Transformation Step Added to Workflow:

Between “Query XML Database” and “Return HTTP Response”, a “Transform Snippet using XSLT” step is introduced. This is where the XSLT processor runs, transforming the XML snippet into a new XML, HTML, or text output as defined by the stylesheet.

3.Response Format May Vary:

The returned HTTP response could be in HTML (for rendering in browsers), plain text, or structured XML, depending on what the XSLT does.

4.XSLT File Management

The web server needs access to a directory or set of predefined XSLT files (for security). The parameter t=transform.xslt should point to a safe, validated XSLT file, avoiding arbitrary external access.

How would the step-by-step description from part (d) need to change?

1. Receive & Parse the HTTP Request

When the web server parses the query string from the URL, the “t” parameter, which specifies the XSLT file to be applied for transformation, would also be extracted.

```
function handleRequest(request):
    filename   = request.queryParams["f"]
    xpathQuery = request.queryParams["q"]
    xsltFile   = request.queryParams.get("t")
```

2. Input Validation

The server would validate “t” (xsltFile) in addition to the filename and xpathQuery.

```
if not isValidFilename(filename):
    return HTTPResponse(400, "Invalid filename")
if not isWellFormedXPath(xpathQuery):
    return HTTPResponse(400, "Invalid XPath")
if xsltFile and not isValidFilename(xsltFile):
    return HTTPResponse(400, "Invalid XSLT filename")
```

Steps 3. Load and Parse the XML, 4. Evaluate the XPath Expression, and 5. Serialise the Resulting Snippet do not need to be altered, as XSLT is not involved. It is only after step 5 that the snippet gets transformed with XSLT in a new sixth step which occurs before the HTTP response is sent.

6. Apply the XSLT Transformation

The server loads the specified XSLT file, applies it to the serialised XML snippet, and transforms the result accordingly. If the XSLT file cannot be found or processing fails, the server returns an appropriate error.

```
if xsltFile:
    try:
        xsltText = readFile("/data/xslt/" + xsltFile)
        transformer = XSLTProcessor(xsltText)
        body = transformer.transform(body)
    except (FileNotFoundError, XSLTError):
        return HTTPResponse(500, "XSLT processing error")
```

7. Send the HTTP Response

It is the same as before, but with the minor adjustment of choosing the appropriate content type based on whether XSLT was applied.

```
contentType = "application/xml; charset=UTF-8"
if xsltFile:
    contentType = determineContentType(xsltFile)

return HTTPResponse(
    status=200,
    headers={"Content-Type": contentType},
    body=body
)
```

Give an example of an appropriate XSLT that might transform a snippet returned by one of your XPath expressions from part (e).

XPath expression:

```
//Dialogue[Character="Mark Grayson"]/Text
```

XSLT that transforms the response of the XPath expression into an HTML fragment listing each spoken line as a bullet point:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

    <xsl:output method="html" indent="yes"/>

    <xsl:template match="/">
        <html>
            <head><title>Mark's Dialogue</title></head>
            <body>
                <h2>Lines spoken by Mark Grayson</h2>
                <ul>
                    <xsl:for-each select="//Text">
                        <li><xsl:value-of select="."/></li>
                    </xsl:for-each>
                </ul>
            </body>
        </html>
    </xsl:template>

</xsl:stylesheet>
```

<xsl:output method="html"/>	Declares the output type as HTML.
<xsl:template match="/">	Matches the root node of the input (which, in this case, is the snippet containing only the <Text> elements).
<xsl:for-each select="//Text">	Iterates over each <Text> node in the snippet.
<xsl:value-of select="."/>	Outputs each text line as an HTML list item.

Describe how that XSLT performs the transformation.

When the XPath expression used is:

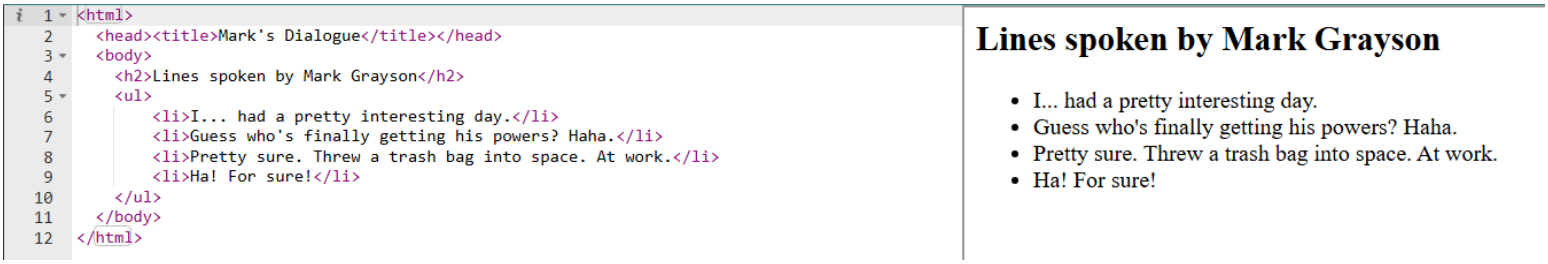
```
//Dialogue[Character="Mark Grayson"]/Text
```

Which returns a list of <Text> nodes spoken by Mark Grayson:

```
<Text>I... had a pretty interesting day.</Text>
<Text>Guess who's finally getting his powers? Haha.</Text>
<Text>Pretty sure. Threw a trash bag into space. At work.</Text>
<Text>Ha! For sure!</Text>
```

Then the transformed HTML returned by the server would be as follows:

```
<html>
  <head><title>Mark's Dialogue</title></head>
  <body>
    <h2>Lines spoken by Mark Grayson</h2>
    <ul>
      <li>I... had a pretty interesting day.</li>
      <li>Guess who's finally getting his powers? Haha.</li>
      <li>Pretty sure. Threw a trash bag into space. At work.</li>
      <li>Ha! For sure!</li>
    </ul>
  </body>
</html>
```



(Screenshot taken in https://html.onlineviewer.net/)

Q2 (a) How humans and machines retrieve information and knowledge from the Web has advanced significantly over the Web's thirty-year history. Discuss this statement in reference to web search, knowledge engineering and advancements in AI. (1200 word limit) [25 Marks]

1. The Evolution of Web Search

In 1990, Archie became the first Internet search engine by indexing FTP archives and allowing users to query filenames, a milestone in retrieval from distributed file repositories [1]. A year later (1991), the Gopher protocol introduced a menu-based client–server system that allowed for hierarchical navigation of text menus and file retrieval [2], acting as a foundation for subsequent web browsers. As the Web grew, AltaVista (1995) pioneered the full-text indexing of HTML pages, enabling searches across document bodies rather than just titles, though its relevance ranking remained limited [3].

The landscape shifted once PageRank was introduced in 1998 [4], an algorithm that treated hyperlinks as weighted votes to rank web pages, improving result relevance and becoming a linchpin of modern search systems. Over time, user-behaviour signals, such as clickthrough and dwell time, were integrated to refine personalisation and contextual relevance of search results.

By 2012, Google launched its Knowledge Graph to represent real-world entities (people, places, concepts) and their relationships, allowing search engines to provide direct, entity-centric answers rather than just lists of URLs [5]. Modern-day search platforms, such as SearchGPT [6], utilise generative AI in concurrence with conventional search engine features to unify content across wikis, and repositories in order to deliver personalised, synthesised results.

2. The Evolution of Knowledge Engineering (& the Semantic Web)

Early attempts to structure web data, such as using HTML meta tags, quickly proved unreliable due to inconsistent adoption and a lack of standardised meaning [7]. To address the challenge of semantic heterogeneity, the field of knowledge engineering introduced ontologies, formal, machine-readable models that define domain entities, their attributes, and their relationships to enable consistent interpretation of domain concepts [8].

Tim Berners-Lee's Semantic Web vision (1999) [9] proposed a global data layer where machines could process content via standards like RDF and OWL, using triples (subject–predicate–object) and rich ontological constructs to interlink and constrain data. Triple stores and OWL reasoners laid the groundwork for knowledge graphs by facilitating inference and data integration across heterogeneous sources.

In practice, large-scale knowledge graphs, such as those by Google and Microsoft, power instant answers, entity linking, and disambiguation within search and their respective digital assistants [5]. Recent advances in ontology learning employ machine learning to extract classes and relations from text bodies, while ontology alignment tools combine independent models for unified querying. Shape Constraint Language (SHACL) further ensures data consistency against ontology schemas, preserving graph integrity [8].

3. The Evolution of Information Retrieval AI

Traditional IR relied on inverted indexes and TF-IDF weighting, which matched queries to documents based on term overlap. Neural IR emerged with models that learn dense representations of queries and documents, capturing semantic similarity beyond keywords [10]. A breakthrough came with the introduction of transformer architecture, first outlined in the 2017 paper "Attention Is All You Need" [11], which dispensed with recurrence and used self-attention to model entire sequences, enabling contextualised embeddings and parallelisable training.

Dense retrieval methods encode queries and passages into a shared vector space and employ approximate nearest-neighbour search (e.g., ANCE) for efficient, semantic retrieval at scale [12]. Retrieval-Augmented Generation (RAG) architectures combine symbolic retrieval with generative language models, grounding outputs in retrieved documents to reduce hallucinations and provide citations [13].

Recent AI models are multimodal, meaning they are capable of processing text, images, audio, and video inputs simultaneously. Rather than just retrieving documents, modern systems are now designed to provide direct answers to user queries. Large Language Models (LLMs) like OpenAI's GPT series can generate fact-based, conversational responses drawn from a broad understanding of web knowledge, bringing information retrieval closer to natural human dialogue.

Conclusion

Over the last thirty years, human–machine information retrieval has shifted from manual browsing and simple keyword matching to AI-powered and semantically aware systems. Web search engines now blend keyword, link, and entity-based ranking with personalised signals; knowledge engineering provides structured frameworks that machines can traverse and reason over; and AI advances, from neural ranking to LLM integration, allow for synthesised, conversational access to knowledge. Looking ahead, integrating real-time knowledge graphs with continual learning models looks to promise more adaptive methods of information retrieval as the technology for it continues to evolve.

References

[1.] A. Emtage et al., "Archie — The First Internet Search Engine," Wikipedia, 1990. [Online]. Available: https://en.wikipedia.org/wiki/Archie_%28search_engine%29

[2.] *Minnesota Computing History*, "Gopher Protocol," 1991. [Online]. Available: https://mncomputinghistory.com/gopher-protocol/

[3.] *Encyclopedia of Information Technology*, "AltaVista," 1995. [Online]. Available: https://en.wikipedia.org/wiki/AltaVista

[4.] L. Page and S. Brin, "The PageRank Citation Ranking: Bringing Order to the Web," in *Proc. 7th Int. World Wide Web Conf. (NeWWW98)*, 1998. [Online]. Available: https://www.cis.upenn.edu/~mkearns/teaching/NetworkedLife/pagerank.pdf

[5.] A. Singhal, "Introducing the Knowledge Graph," *Google Blog*, 2012. [Online]. Available: https://blog.google/products/search/introducing-knowledge-graph-things-not/

[6.] *Wikipedia Contributors*, "Timeline of web search engines," *Wikipedia*, Nov. 18, 2019. [Online]. Available: https://en.wikipedia.org/wiki/Timeline_of_web_search_engines
[7.] R. Guns, "Tracing the origins of the semantic web," *J. Amer. Soc. Inf. Sci. Technol.*, vol. 64, no. 10, pp. 2173–2181, Aug. 2013, doi: 10.1002/asi.22907.
[8.] N. F. Noy, "A Knowledge Engineering Approach to Develop Domain Ontology," *SIKS Dissertation Series*, 2019. [Online]. Available: https://www.researchgate.net/publication/220233110_A_Knowledge_Engineering_Approach_to_Develop_Domain_Ontology
[9.] T. Berners-Lee, "The Semantic Web," *Scientific American*, 2001. [Online]. Available: https://en.wikipedia.org/wiki/Semantic_Web
[10.] S. Kumar, "Large Language Models for Information Retrieval: A Survey," *arXiv preprint arXiv:2308.07107*, 2023. [Online]. Available: https://arxiv.org/html/2308.07107v3
[11.] A. Vaswani *et al.*, "Attention Is All You Need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017. [Online]. Available: https://papers.nips.cc/paper/7181-attention-is-all-you-need
[12.] L. Xiong *et al.*, "ANCE: Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval," *arXiv preprint arXiv:2007.00808*, 2020. [Online]. Available: https://arxiv.org/abs/2007.00808
[13.] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

(b) Model an Ontology that represents concepts and relationships derived and extended from the XSD created in Q1 (b). The Ontology should include;
>appropriate classes (at least 10),
>their hierarchy,
>properties/relationships (both object and data; at least 15)
>and axioms (at least 2).
Discuss the rationale behind your modelling of these. The Ontology may be created in Protégé or similar tool. Screenshots should be used to illustrate your answer. [20 Marks]

- ***Classes***

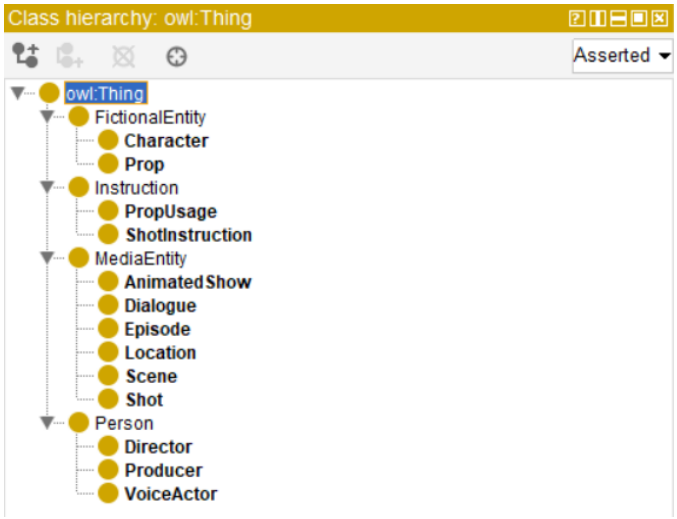
The classes selected were designed to represent the key concepts involved in the production process for the chosen show. Each class was selected based on its necessity to describe the real-world entities, actions, and responsibilities captured by the original XSD schema.

Class	Description
AnimatedShow	A programme produced by means of animation (root).
Episode	An instalment of a show.
Location	The place where a scene takes place
Scene	A collection of shots within the same location.
Shot	A change in camera position within a scene.
ShotInstruction	Description of what happens in a shot.
Prop	Object required to be animated in a scene.
PropUsage	Description of what the object is used for.
Character	Fictional entity that appears in an episode.
Dialogue	Spoken line(s) in a shot.
VoiceActor	A real person voicing a character.
Director	Oversees all aspects of the animation process.
Producer	Oversees all departments working on the show.

The classes selected for the Invincible ontology were chosen to comprehensively represent both the fictional content of the animated show and the real-world production elements behind it. The “AnimatedShow” class serves as the root, encapsulating the entire system, while “Episode” structures the show into distinct instalments. Within each episode, “Scene” and “Shot” organise the narrative spatially and visually, with “Location” providing the physical context. “ShotInstruction” captures specific guidance for the animation team on camera movements or character actions, while “Prop” and “PropUsage” model the use of in-scene objects, justifying the introduction of a general “Instruction” superclass (see Hierarchy). To represent characters and dialogue, the ontology distinguishes between the fictional “Character” and the real-world “VoiceActor”, reflecting the separation of story elements and production roles. Additionally, “Director” and “Producer” classes were included to model the creative and organisational leadership vital to episode and show production.

➢ *Class Hierarchy*

The class hierarchy organises the previously mentioned classes into groups of subclasses, each under their own superclass, to reflect the different roles played within the show's production and narrative structure. Following OWL standards, at the highest level, everything falls under “owl:Thing”.

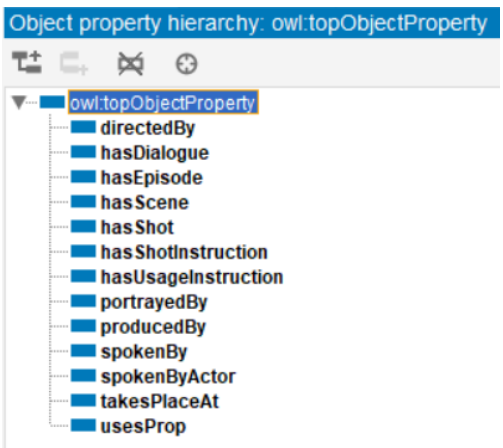


Fictional elements like characters and props are separated under the “FictionalEntity”, while real-world contributors are grouped under “Person”. Instructions for animation, whether for shots or props, are unified under “Instruction”. Media-related elements like shows, episodes, scenes, shots, and dialogue are grouped under “MediaEntity” to reflect their narrative role. Although location could be placed under “FictionalEntity” due to the show being a work of fiction or its own “Place” superclass, upon further consideration it was put under “MediaEntity”, as it is heavily tied to recognising each new scene (a change in location).

➢ *Properties / Relationships*

The object and data properties define the interactions and attributes of the classes within the ontology, enabling a structured and meaningful connection between entities. Following OWL standards, object properties describe relationships between two individuals (instances of classes), while data properties link individuals to literal values such as numbers or text.

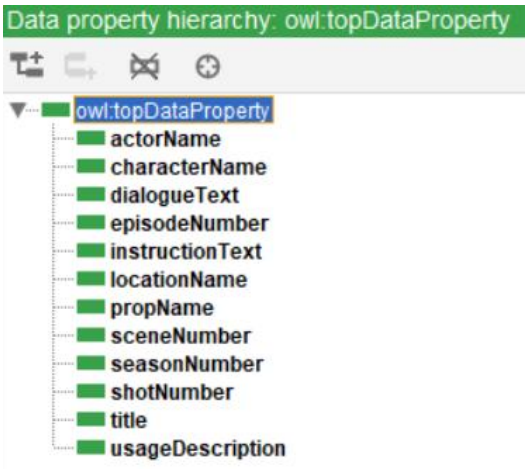
Object Properties



Object Properties		
Property	Domain >>> Range	Relationship
directedBy	Episode >>> Person	Episode directed by Person
hasDialogue	Shot >>> Dialogue	Shot contains Dialogues
hasEpisode	AnimatedShow >>> Episode	Show contains Episodes
hasScene	Episode >>> Scene	Episode contains Scenes
hasShot	Scene >>> Shot	Scene contains Shots
hasShotInstruction	Shot >>> ShotInstruction	Shot has Instructions
hasUsageInstruction	Prop >>> PropUsage	Prop has Usage Instructions
portrayedBy	Character >>> VoiceActor	Character is portrayed by a Voice Actor
producedBy	Episode >>> Person	Episode produced by Person
spokenBy	Dialogue >>> Character	Dialogue is spoken by a Character
spokenByActor	Dialogue >>> VoiceActor	Dialogue is inferred (see Axioms) to be spoken by a Voice Actor
takesPlaceAt	Scene >>> Location	Scene takes place at a Location
usesProp	Shot >>> Prop	Shot uses Props

The object properties in the ontology portray the relationships shared between each class in the show. Properties like “hasEpisode”, “hasScene”, and “hasShot” model the narrative hierarchy, associating the show to its episodes, episodes to their scenes, and scenes to shots that they are comprised of. Similarly, “hasDialogue” and “hasShotInstruction” ensure that elements such as spoken lines and shot descriptions correlate with the appropriate shot. Character interaction is modelled through “spokenBy” (linking dialogue to characters) and “portrayedBy” (linking characters to their voice actors). There is also an inferred connection between dialogue and voice actors through “spokenByActor” (covered further in Axioms). Production roles are integrated with “directedBy” and “producedBy” to connect episodes to directors and producers, reflecting the behind-the-scenes creative control. Additionally, “usesProp” captures the use of physical objects within shots, and “takesPlaceAt” links scenes to their locations.

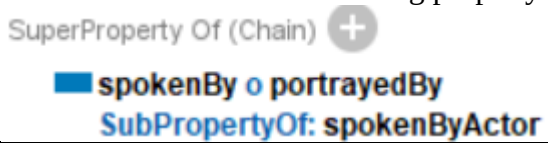
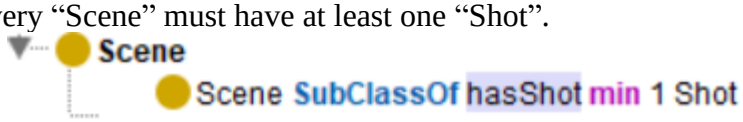
Data Properties



Data Properties			
Property	Domain	Datatype	Example
actorName	VoiceActor	string	“J.K. Simmons”
characterName	Character	string	“Nolan Grayson”
dialogueText	Dialogue	string	“Sorry I'm late. Honest to…”
episodeNumber	Episode	integer	1
instructionText	ShotInstruction	string	“Nolan arrives at the…”
locationName	Location	string	"Grayson Family Dining Room"
propName	Prop	string	“Wine Glass”
sceneNumber	Scene	integer	5
seasonNumber	AnimatedShow	integer	1
shotNumber	Shot	integer	4
title	AnimatedShow/Episode	string	“Invincible”
usageDescription	Prop	string	“Almost tips over as…”

The data properties serve to assign attributes to each class in the ontology. Properties like “title”, “seasonNumber”, and “episodeNumber” are used to identify and organise each episode in the show. “sceneNumber” and “shotNumber” aid in tracking the structure within each episode. To assist animators and voice actors in interpreting the narrative of a scene, “instructionText” annotates each shot instruction, while “dialogueText” records the spoken content of dialogue lines. “propName” and “usageDescription” document the non-character entities used in scenes and their intended functions. Furthermore, “characterName”, “actorName”, and “locationName” provide basic identifiers for fictional and real-world elements involved in production.

- ***Axioms***

Axiom	Description
Dialogue-VoiceActor Inference via Character	“Dialogue” is spoken by a “Character”. Since that “Character” is portrayed by a “VoiceActor”, then that “Dialogue” can be indirectly linked to the “VoiceActor” using property chains: 
Scene-Shot Cardinality Constraint	Every “Scene” must have at least one “Shot”. 

(Screenshots taken in Protégé)

(c) Illustrate how relationships between objects may be represented in Resource Description Framework using a snippet of RDF (XML Syntax). [5 Marks]

```
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:inv="http://example.org/invincible#">

    <!-- Scene 5 -->
    <inv:Scene rdf:about="http://example.org/invincible/Scene5">
        <inv:hasSceneNumber>5</inv:hasSceneNumber>
        <inv:hasDescription>The Grayson family sits at the dinner table. Mark is about to tell his
parents the good news.</inv:hasDescription>
        <inv:hasLocation rdf:resource="http://example.org/invincible/GraysonDiningRoom"/>
        <inv:hasShot rdf:resource="http://example.org/invincible/Scene5/Shot1"/>
    </inv:Scene>

    <!-- Shot 1 -->
    <inv:Shot rdf:about="http://example.org/invincible/Scene5/Shot1">
        <inv:hasShotNumber>1</inv:hasShotNumber>
        <inv:hasShotInstructions>Mark enters the dining room and sits at the table. Debbie finishes
setting the table.</inv:hasShotInstructions>
        <inv:hasProp rdf:resource="http://example.org/invincible/PlateOfRoastChicken"/>
        <inv:hasDialogue rdf:resource="http://example.org/invincible/Dialogue1"/>
    </inv:Shot>

    <!-- Prop (Plate of Roast Chicken) -->
    <inv:Prop rdf:about="http://example.org/invincible/PlateOfRoastChicken">
        <inv:hasPropName>Plate of Roast Chicken</inv:hasPropName>
        <inv:hasUsage>Held by Debbie. Placed on the table out of view for the rest of the
scene.</inv:hasUsage>
    </inv:Prop>

    <!-- Dialogue -->
    <inv:Dialogue rdf:about="http://example.org/invincible/Dialogue1">
        <inv:spokenBy rdf:resource="http://example.org/invincible/DebbieGrayson"/>
        <inv:hasText>And a big welcome home to my favorite son.</inv:hasText>
    </inv:Dialogue>

    <!-- Character -->
    <inv:Character rdf:about="http://example.org/invincible/DebbieGrayson">
        <inv:hasCharacterName>Debbie Grayson</inv:hasCharacterName>
    </inv:Character>

</rdf:RDF>
```

Appendix

A1. [2(b) – Invincible.OWL]

```
Prefix(:=<http://www.example.org/invincible#>)

Prefix(owl:=<http://www.w3.org/2002/07/owl#>)

Prefix(rdf:=<http://www.w3.org/1999/02/22-rdf-syntax-ns#>)

Prefix(xml:=<http://www.w3.org/XML/1998/namespace>)

Prefix(xsd:=<http://www.w3.org/2001/XMLSchema#>)

Prefix(rdfs:=<http://www.w3.org/2000/01/rdf-schema#>)



Ontology(<http://www.example.org/invincible>

Declaration(Class(:AnimatedShow))

Declaration(Class(:Character))

Declaration(Class(:Dialogue))

Declaration(Class(:Director))

Declaration(Class(:Episode))

Declaration(Class(:FictionalEntity))

Declaration(Class(:Instruction))

Declaration(Class(:Location))

Declaration(Class(:MediaEntity))

Declaration(Class(:Person))

Declaration(Class(:Producer))

Declaration(Class(:Prop))

Declaration(Class(:PropUsage))

Declaration(Class(:Scene))

Declaration(Class(:Shot))

Declaration(Class(:ShotInstruction))

Declaration(Class(:VoiceActor))

Declaration(ObjectProperty(:directedBy))

Declaration(ObjectProperty(:hasDialogue))

Declaration(ObjectProperty(:hasEpisode))

Declaration(ObjectProperty(:hasScene))

Declaration(ObjectProperty(:hasShot))

Declaration(ObjectProperty(:hasShotInstruction))

Declaration(ObjectProperty(:hasUsageInstruction))

Declaration(ObjectProperty(:portrayedBy))

Declaration(ObjectProperty(:producedBy))

Declaration(ObjectProperty(:spokenBy))

Declaration(ObjectProperty(:spokenByActor))

Declaration(ObjectProperty(:takesPlaceAt))

Declaration(ObjectProperty(:usesProp))

Declaration(DataProperty(:actorName))

Declaration(DataProperty(:characterName))

Declaration(DataProperty(:dialogueText))

Declaration(DataProperty(:episodeNumber))

Declaration(DataProperty(:instructionText))

Declaration(DataProperty(:locationName))

Declaration(DataProperty(:propName))
```

```
Declaration(DataProperty(:sceneNumber))

Declaration(DataProperty(:seasonNumber))

Declaration(DataProperty(:shotNumber))

Declaration(DataProperty(:title))

Declaration(DataProperty(:usageDescription))

#############################

#   Object Properties

#############################


# Object Property: :directedBy (:directedBy)


ObjectPropertyDomain(:directedBy :Episode)

ObjectPropertyRange(:directedBy :Director)


# Object Property: :hasDialogue (:hasDialogue)


ObjectPropertyDomain(:hasDialogue :Shot)

ObjectPropertyRange(:hasDialogue :Dialogue)


# Object Property: :hasEpisode (:hasEpisode)


ObjectPropertyDomain(:hasEpisode :AnimatedShow)

ObjectPropertyRange(:hasEpisode :Episode)


# Object Property: :hasScene (:hasScene)


ObjectPropertyDomain(:hasScene :Episode)

ObjectPropertyRange(:hasScene :Scene)


# Object Property: :hasShot (:hasShot)


ObjectPropertyDomain(:hasShot :Scene)

ObjectPropertyRange(:hasShot :Shot)


# Object Property: :hasShotInstruction (:hasShotInstruction)


ObjectPropertyDomain(:hasShotInstruction :Shot)

ObjectPropertyRange(:hasShotInstruction :ShotInstruction)


# Object Property: :hasUsageInstruction (:hasUsageInstruction)


ObjectPropertyDomain(:hasUsageInstruction :Prop)

ObjectPropertyRange(:hasUsageInstruction :PropUsage)


# Object Property: :portrayedBy (:portrayedBy)


ObjectPropertyDomain(:portrayedBy :Character)

ObjectPropertyRange(:portrayedBy :VoiceActor)
```

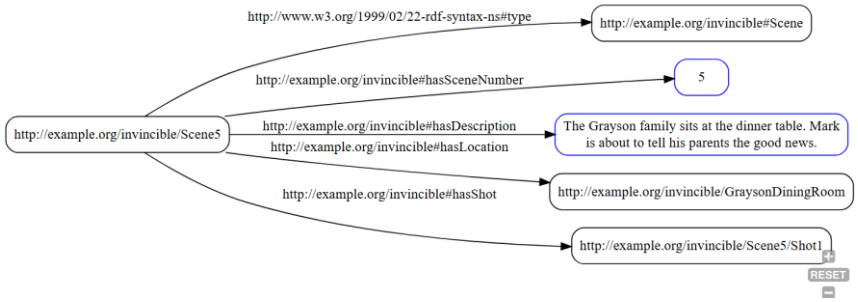
```
# Object Property: :producedBy (:producedBy)

ObjectPropertyDomain(:producedBy :Episode)
ObjectPropertyRange(:producedBy :Producer)

# Object Property: :spokenBy (:spokenBy)

ObjectPropertyDomain(:spokenBy :Dialogue)
ObjectPropertyRange(:spokenBy :Character)

# Object Property: :spokenByActor (:spokenByActor)

ObjectPropertyDomain(:spokenByActor :Dialogue)
ObjectPropertyRange(:spokenByActor :VoiceActor)

# Object Property: :takesPlaceAt (:takesPlaceAt)

ObjectPropertyDomain(:takesPlaceAt :Scene)
ObjectPropertyRange(:takesPlaceAt :Location)

# Object Property: :usesProp (:usesProp)

ObjectPropertyDomain(:usesProp :Shot)
ObjectPropertyRange(:usesProp :Prop)


#############################
#   Data Properties
#############################

# Data Property: :actorName (:actorName)

DataPropertyDomain(:actorName :VoiceActor)
DataPropertyRange(:actorName xsd:string)

# Data Property: :characterName (:characterName)

DataPropertyDomain(:characterName :Character)
DataPropertyRange(:characterName xsd:string)

# Data Property: :dialogueText (:dialogueText)

DataPropertyDomain(:dialogueText :Dialogue)
DataPropertyRange(:dialogueText xsd:string)

# Data Property: :episodeNumber (:episodeNumber)

DataPropertyDomain(:episodeNumber :Episode)
```

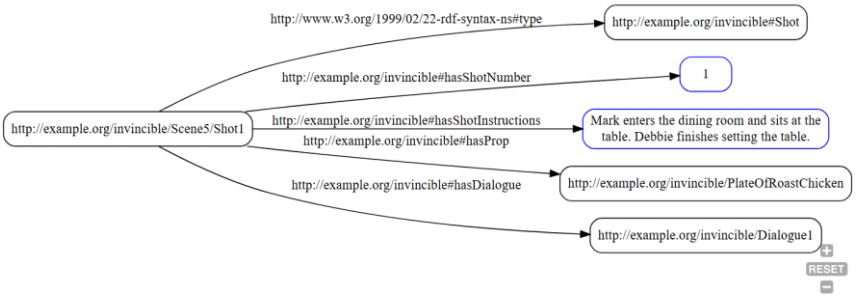
```
DataPropertyRange(:episodeNumber xsd:integer)


# Data Property: :instructionText (:instructionText)


DataPropertyDomain(:instructionText :ShotInstruction)

DataPropertyRange(:instructionText xsd:string)


# Data Property: :locationName (:locationName)


DataPropertyDomain(:locationName :Location)

DataPropertyRange(:locationName xsd:string)


# Data Property: :propName (:propName)


DataPropertyDomain(:propName :Prop)

DataPropertyRange(:propName xsd:string)


# Data Property: :sceneNumber (:sceneNumber)


DataPropertyDomain(:sceneNumber :Scene)

DataPropertyRange(:sceneNumber xsd:integer)


# Data Property: :seasonNumber (:seasonNumber)


DataPropertyDomain(:seasonNumber :AnimatedShow)

DataPropertyRange(:seasonNumber xsd:integer)


# Data Property: :shotNumber (:shotNumber)


DataPropertyDomain(:shotNumber :Shot)

DataPropertyRange(:shotNumber xsd:integer)


# Data Property: :title (:title)


DataPropertyDomain(:title :AnimatedShow)

DataPropertyDomain(:title :Episode)

DataPropertyRange(:title xsd:string)


# Data Property: :usageDescription (:usageDescription)


DataPropertyDomain(:usageDescription :Prop)

DataPropertyRange(:usageDescription xsd:string)




#############################

#   Classes

#############################
```

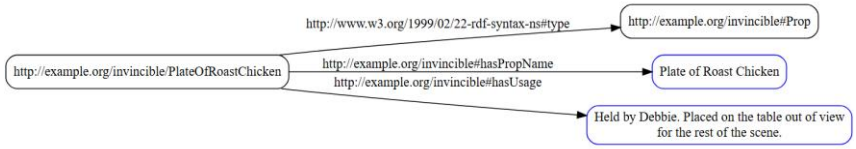
```
# Class: :AnimatedShow (:AnimatedShow)

SubClassOf(:AnimatedShow :MediaEntity)

# Class: :Character (:Character)

SubClassOf(:Character :FictionalEntity)

# Class: :Dialogue (:Dialogue)

SubClassOf(:Dialogue :MediaEntity)

# Class: :Director (:Director)

SubClassOf(:Director :Person)

# Class: :Episode (:Episode)

SubClassOf(:Episode :MediaEntity)

# Class: :Location (:Location)

SubClassOf(:Location :MediaEntity)

# Class: :Producer (:Producer)

SubClassOf(:Producer :Person)

# Class: :Prop (:Prop)

SubClassOf(:Prop :FictionalEntity)

# Class: :PropUsage (:PropUsage)

SubClassOf(:PropUsage :Instruction)

# Class: :Scene (:Scene)

SubClassOf(:Scene :MediaEntity)
SubClassOf(:Scene ObjectMinCardinality(1 :hasShot :Shot))

# Class: :Shot (:Shot)

SubClassOf(:Shot :MediaEntity)

# Class: :ShotInstruction (:ShotInstruction)

SubClassOf(:ShotInstruction :Instruction)
```

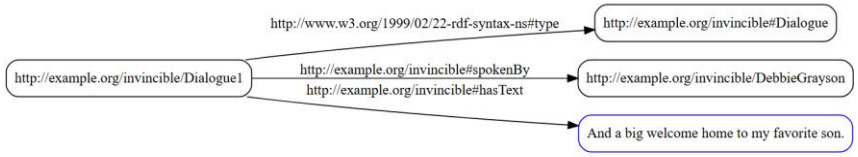
```
# Class: :VoiceActor (:VoiceActor)


SubClassOf(:VoiceActor :Person)



SubObjectPropertyOf(ObjectPropertyChain(:spokenBy :portrayedBy) :spokenByActor)
)
```

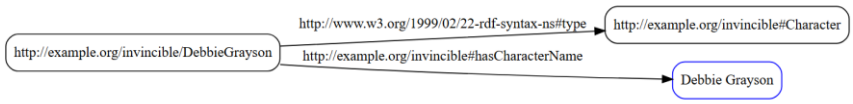
A2. [2(c) – RDF Snippet Screenshots]











(Screenshots taken in https://semantechs.co.uk/turtle-editor-viewer/)